

Cloud Computing

Mathematical Engineering and Artificial Intelligence

Lab Practice 1:

Basic concepts and creation of virtual machines for machine learning

Lab Report

Authors:

Jorge Carniero Príncipe
Andrés Gil Vicente

Date:

14/09/2026

Contents

1	Objective	3
2	Task 1 – Compute Engine instance for machine learning	3
2.1	Existing VPCs and custom network	3
2.2	Firewall rules	5
2.3	VM1	7
2.4	Cloud Shell and SSH tunnel	9
2.5	JupyterLab	10
3	Task 2 – Cloud Storage	11
3.1	Cloud Shell configuration	11
3.2	Bucket creation	11
3.3	File upload	12
3.4	Browser access and permissions	12
4	Task 3 – Custom images	13
4.1	VM2	13
4.2	Nginx installation	13
4.3	Browser verification and HTTP firewall rule	14
4.4	Custom image	15
4.5	VM3	15
4.6	Cleanup	16
5	Task 4 – Google Cloud Marketplace	17
6	Task 5 – Google Cloud CLI	18
6.1	Environment variables	18
6.2	Create VM2	19
6.3	Install and verify Nginx	19
6.4	Stop VM2 and create the image	20
6.5	Delete VM2	20
6.6	Create VM3 from the custom image	21
6.7	Verify VM3	21

6.8 Cleanup	22
7 Task 6 – Google Cloud API	23
8 Conclusions	23

1 Objective

This lab introduces the basic building blocks needed to run machine learning workloads on Google Cloud: creating a custom VPC network with its own subnet, defining firewall rules, launching Compute Engine instances (including a Deep Learning VM running JupyterLab through an SSH tunnel), creating and using a Cloud Storage bucket, building and reusing a custom VM image, and repeating the same work from the Google Cloud CLI and later, from the Google Cloud API/SDK.

As required by the statement, each of us worked with his own Google account and his own project: `lab1-a` (ID `commanding-port-508205-k5`) and (`modern-sublime-508206-i7`). Everything was done in region `eu-west-1`. The two runs are reported together and the screenshots come from the first project, so that resource names and IP addresses stay consistent through the document.

2 Task 1 – Compute Engine instance for machine learning

2.1 Existing VPCs and custom network

Q: Are there any already created VPCs? Which one? In what regions is there a subnet?

Yes. The project already contained the VPC called `default`, created automatically with the project. It is an auto-mode network, so it has one subnet in each available Compute Engine region, 42 in total.

Regions: `africa-south1`, `asia-east1`, `asia-east2`, `asia-northeast1`, `asia-northeast2`, `asia-northeast3`, `asia-south1`, `asia-south2`, `asia-southeast1`, `asia-southeast2`, `asia-southeast3`, `australia-southeast1`, `australia-southeast2`, `eu-central1`, `eu-north1`, `eu-north2`, `eu-southwest1`, `eu-west1`, `eu-west10`, `eu-west12`, `eu-west2`, `eu-west3`, `eu-west4`, `eu-west6`, `eu-west8`, `eu-west9`, `me-central1`, `me-west1`, `na-northeast1`, `na-northeast2`, `na-south1`, `sa-east1`, `sa-west1`, `us-central1`, `us-east1`, `us-east4`, `us-east5`, `us-south1`, `us-west1`, `us-west2`, `us-west3`, `us-west4`.

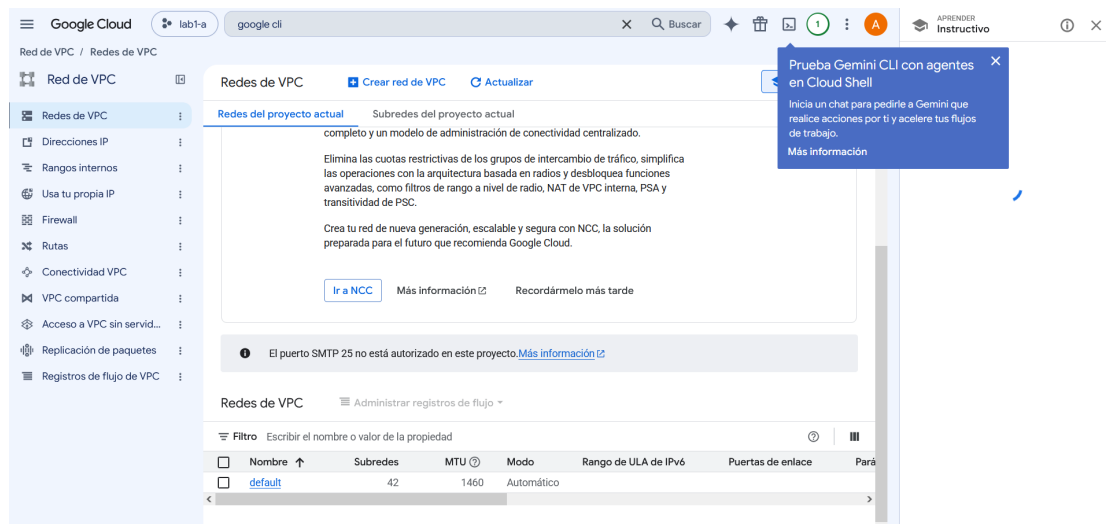


Figure 1: Default VPC already present in the project, with 42 subnets

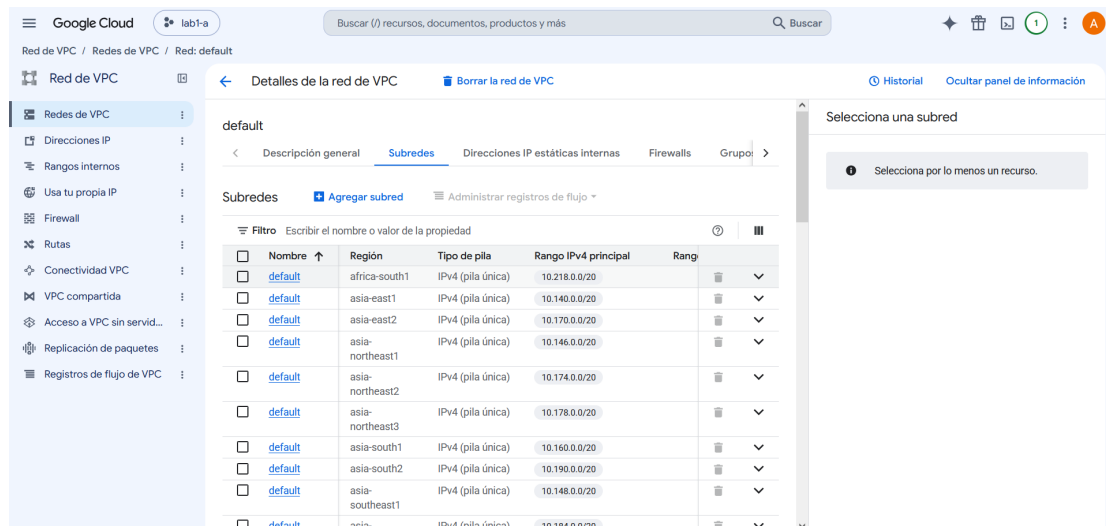
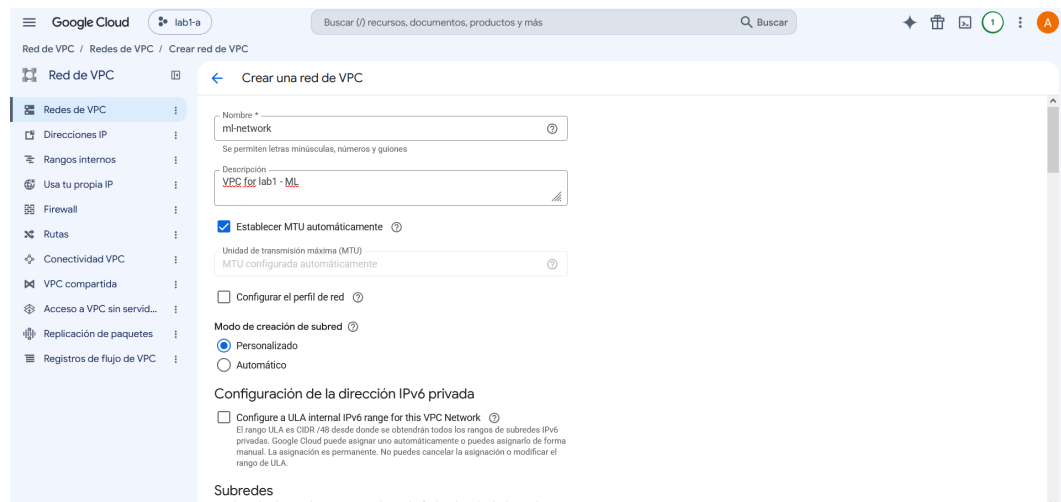
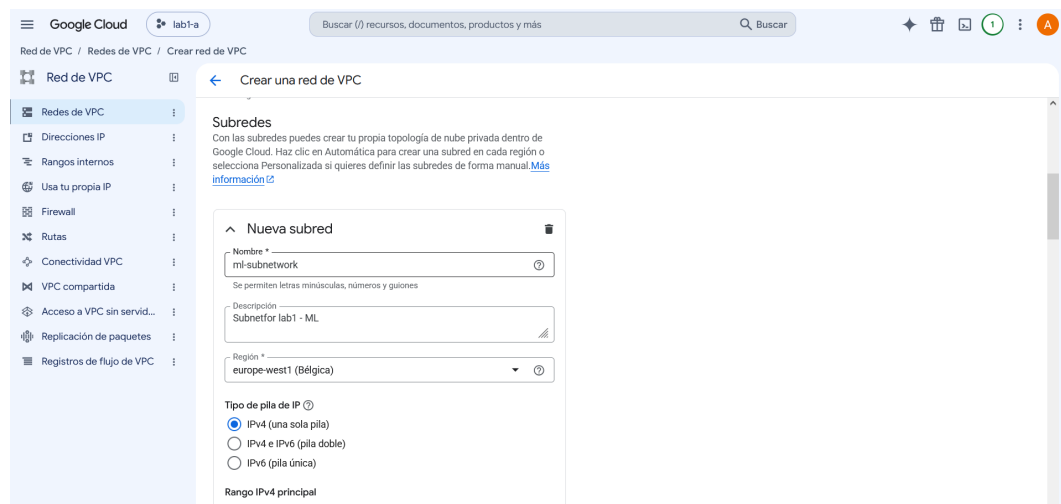


Figure 2: Subnets of the default VPC, one per region

A new VPC named `ml-network` was then created in *custom* creation mode, with a single subnet `ml-subnetwork` in `eu-west-1` and range `10.0.0.0/24`.



(a) Network in custom creation mode



(b) Subnet in europe-west1

Figure 3: Creation of the custom VPC ml-network

2.2 Firewall rules

Q: What are the default firewall rules? What are the proposed firewall rules? Do you see any issue with the proposed firewall rules?

Every VPC has two implied rules that cannot be selected or removed: deny all ingress traffic and allow all egress traffic. Also, the console proposes four optional rules.

Rule	Protocol / port	Source range	Priority
ml-network-deny-all-ingress	all	0.0.0.0/0	65535
ml-network-allow-all-egress	all	0.0.0.0/0	65535
ml-network-allow-custom	all	10.0.0.0/24	65534
ml-network-allow-icmp	ICMP	0.0.0.0/0	65534
ml-network-allow-ssh	TCP 22	0.0.0.0/0	65534
ml-network-allow-rdp	TCP 3389	0.0.0.0/0	65534

Table 1: Implied rules (first two) and rules proposed by the console

The problem is that three of the proposed rules (ICMP, SSH and RDP) accept traffic from 0.0.0.0/0, that is, from anywhere on the Internet. The most important ones are SSH and RDP, which give direct administrative access to the instances.

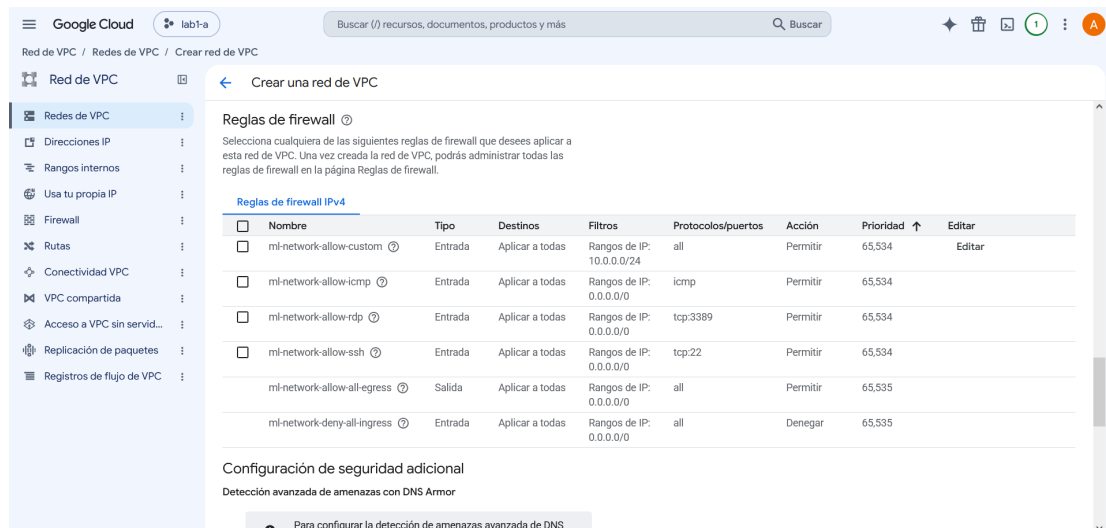


Figure 4: Firewall rules proposed by the console (none of them was selected)

Instead, a single ingress rule was created to restrict SSH to Google's Identity-Aware Proxy range:

Name	ml-network-allow-ssh-from-iap
Network	ml-network
Targets	All instances in the network
Source ranges	35.235.240.0/20
Protocol/port	TCP 22
Action	Allow (ingress), priority 1000

Table 2: Firewall rule used for SSH access

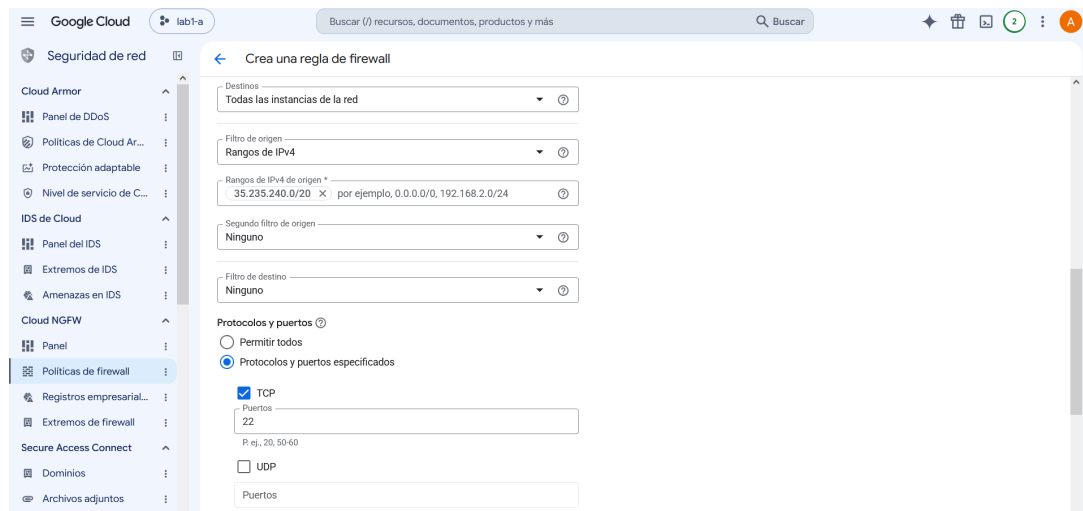


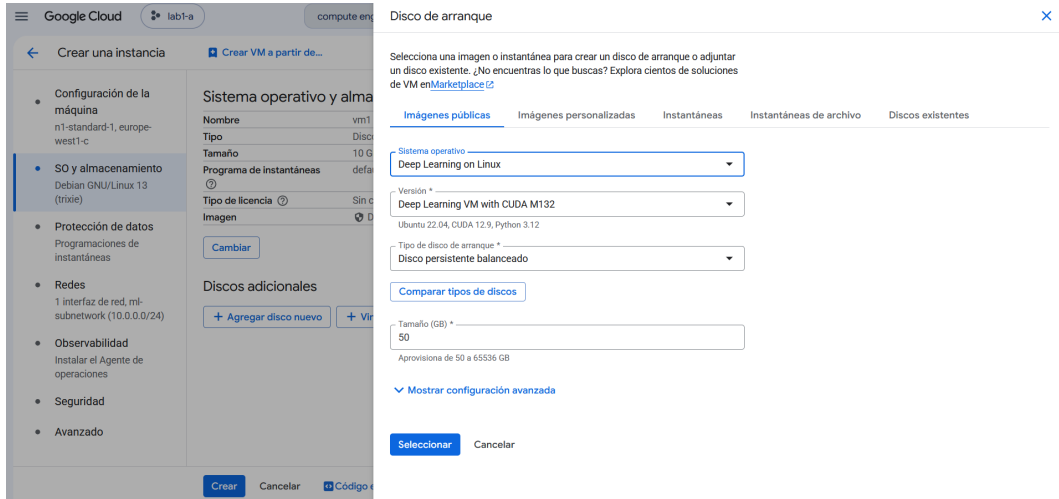
Figure 5: Source range and port of `ml-network-allow-ssh-from-iap`

2.3 VM1

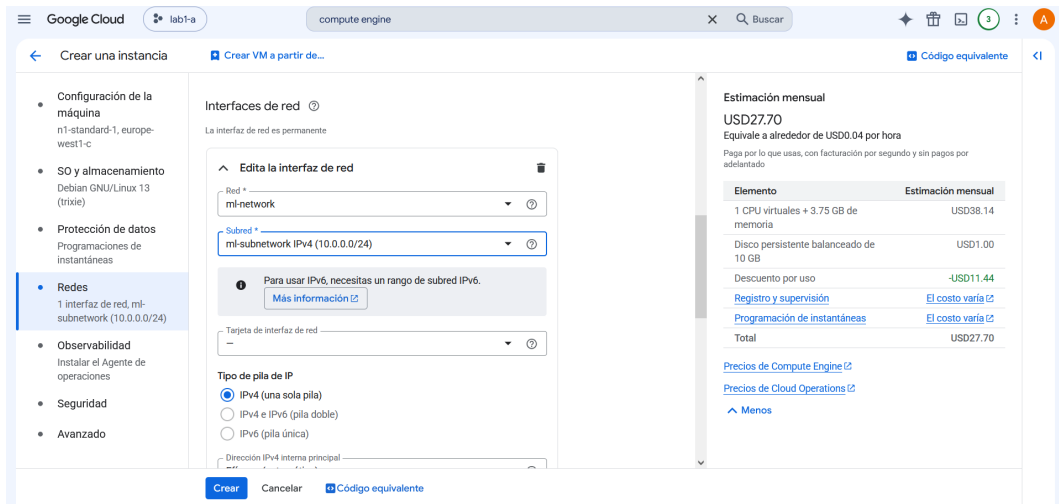
The first Compute Engine instance was created with the configuration below and attached to `ml-network`.

Name	<code>vm1</code>
Machine type	<code>n1-standard-1</code>
Region / zone	<code>europa-west1</code> / <code>europa-west1-c</code>
Image	Deep Learning VM with CUDA M132
Boot disk	Balanced persistent disk, 50 GB
Network tag	<code>deep</code>
Network / subnet	<code>ml-network</code> / <code>ml-subnetwork</code> (10.0.0.0/24)

Table 3: VM1 configuration



(a) Deep Learning VM image selected for the boot disk



(b) Network interface attached to ml-network

Figure 6: VM1 configuration

The same image and machine type were used in both projects. We connected to the instance with the browser SSH option in the console. The versions to be reported were those of the operating system, PyTorch and NumPy instead of the full list given in the statement; Python and JupyterLab were checked as well.

Component	Version
Operating system	Ubuntu 22.04.5 LTS (Jammy Jellyfish)
PyTorch	2.14.0+cu130
NumPy	2.2.6
Python	3.10.12
JupyterLab	4.6.3

Table 4: Software versions reported from VM1

```
andres_cloud_imat@vm1:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
andres_cloud_imat@vm1:~$ python3 -c "import numpy; print(numpy.__version__)"
2.2.6
andres_cloud_imat@vm1:~$ python3 -c "import torch; print(torch.__version__)"
2.14.0+cu130
andres_cloud_imat@vm1:~$ python3 --version
Python 3.10.12
andres_cloud_imat@vm1:~$ jupyter lab --version
4.6.3
andres_cloud_imat@vm1:~$
```

Figure 7: Versions checked inside VM1

2.4 Cloud Shell and SSH tunnel

After opening Cloud Shell, the project and the zone were set:

Listing 2.1: Cloud Shell project and zone

```
gcloud config set project commanding-port-508205-k5
gcloud config set compute/zone europe-west1-c
```

The command given in the statement did not work as it was, because SSH is only allowed from the IAP range, so the `-tunnel-through-iap` flag had to be added:

Listing 2.2: SSH tunnel to VM1 through IAP

```
gcloud compute ssh vm1 \
  --zone=europe-west1-c \
  --tunnel-through-iap \
  -- -L 8080:localhost:8080
```

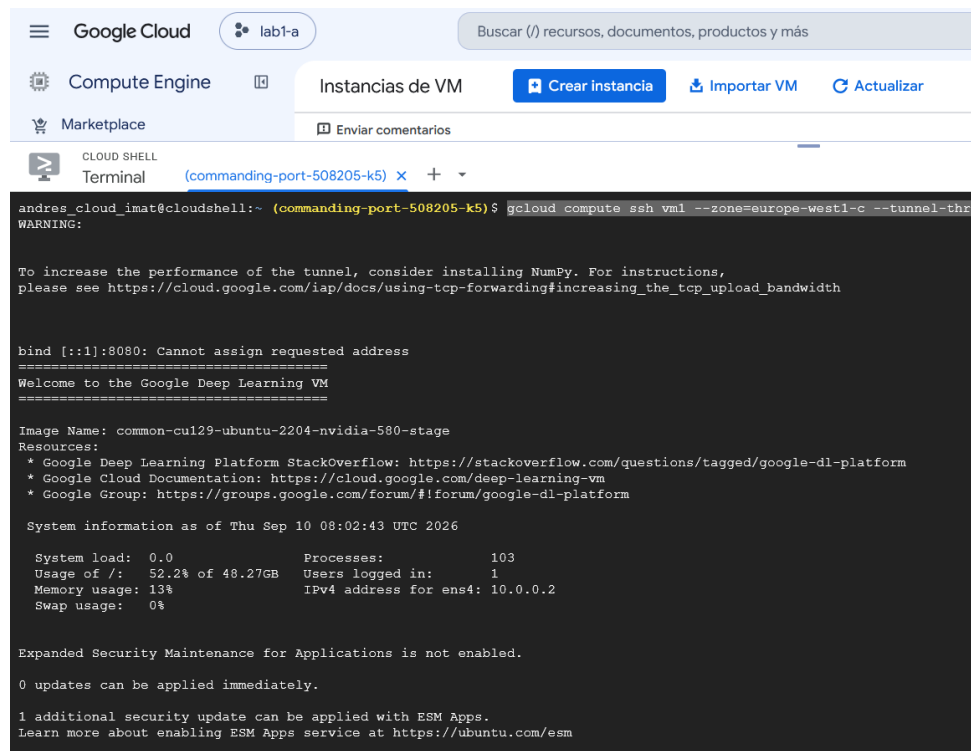


Figure 8: SSH session to VM1 through the IAP tunnel

2.5 JupyterLab

JupyterLab was started inside VM1 listening on port 8080:

Listing 2.3: Starting JupyterLab inside VM1

```
python3 -m jupyterlab --ip=0.0.0.0 --port=8080 --no-browser --allow-root
  ↳ --ServerApp.token='' --ServerApp.password='' --ServerApp.
  ↳ allow_origin='*' --ServerApp.allow_remote_access=True --ServerApp.
  ↳ trust_xheaders=True --ServerApp.disable_check_xsrf=True
```

```

andres@cloud_ima81080:~$ python3 on_jupyterlab --app=0.0.0 --port=8080 --no-browser --allow-root --ServerApp.token="" --ServerApp.password="" --ServerApp.allow_origin="" --ServerApp.allow_root_token="" --ServerApp.trust_xheaders=True --ServerApp.disable_check_xheader=True
[W 2026-09-10 08:00:35.139 ServerApp] ServerApp.token config is deprecated in 2.0. Use IdentityProvider.token.
[I 2026-09-10 08:00:35.224 ServerApp] JupyterLab | extension was successfully linked.
[I 2026-09-10 08:00:35.234 ServerApp] JupyterLab | extension was successfully linked.
[I 2026-09-10 08:00:35.234 ServerApp] JupyterLab | extension was successfully linked.
[I 2026-09-10 08:00:35.243 ServerApp] notebook | extension was successfully linked.
[I 2026-09-10 08:00:35.243 ServerApp] Writing Jupyter server cookie secret to /home/andres.cloud_ima81080/.local/share/jupyter/runtime/jupyter_cookie_secret
[I 2026-09-10 08:00:35.261 ServerApp] notebook_shim | extension was successfully linked.
[W 2026-09-10 08:00:35.311 ServerApp] All authentication is disabled. Anyone who can connect to this server will be able to run code.
[I 2026-09-10 08:00:35.312 ServerApp] notebook_shim | extension was successfully loaded.
[I 2026-09-10 08:00:35.315 ServerApp] JupyterLab | extension was successfully loaded.
[I 2026-09-10 08:00:35.327 ServerApp] JupyterLab | extension was successfully loaded.
[I 2026-09-10 08:00:35.329 ServerApp] JupyterLab | extension loaded from /home/andres.cloud_ima81080/.local/lib/python3.10/site-packages/jupyterlab
[I 2026-09-10 08:00:35.330 LabApp] JupyterLab application directory is /home/andres.cloud_ima81080/.local/share/jupyter/lab
[I 2026-09-10 08:00:35.321 LabApp] Extension Manager is 'ppyl'.
[I 2026-09-10 08:00:35.321 ServerApp] JupyterLab | extension was successfully loaded.
[I 2026-09-10 08:00:35.330 ServerApp] notebook | extension was successfully loaded.
[I 2026-09-10 08:00:35.331 ServerApp] Serving notebooks from local directory: /home/andres.cloud_ima81080
[I 2026-09-10 08:00:35.331 ServerApp] Jupyter Server 2.21.0 is running at
[I 2026-09-10 08:00:35.331 ServerApp] http://0.0.0.0:8080/lab
[I 2026-09-10 08:00:35.331 ServerApp] http://127.0.0.1:8080/lab
[I 2026-09-10 08:00:35.341 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[I 2026-09-10 08:00:35.341 ServerApp] Skipped non-installed server(s): basepython, bash-language-server, dockstore-language-server, nodejs, javascript-typescript-languageserver, jedi-language-server, julia-language-server, kotlin-language-server, lua-language-server, perl-language-server, python-language-server, texlab, typescript-language-server, unified-language-server, vscode-languagesserver-bin, vscode-javascript-language-server, vscode-javascript-typescript-languagesserver-bin, yamllanguage-server
deeps@language-server-bin:~$

```

Figure 9: Starting JupyterLab on port 8080

With the tunnel active, the interface was opened from Cloud Shell using *Web Preview* on port 8080.

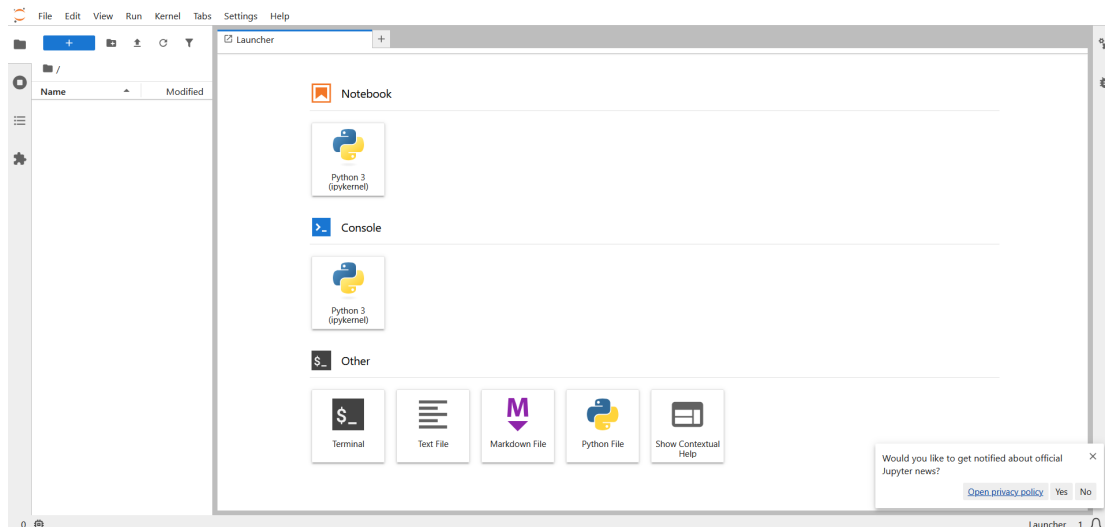


Figure 10: JupyterLab opened through the Cloud Shell Web Preview

Q: What is the installed version of JupyterLab?

JupyterLab 4.6.3.

3 Task 2 – Cloud Storage

3.1 Cloud Shell configuration

The current configuration was checked and the default project was set:

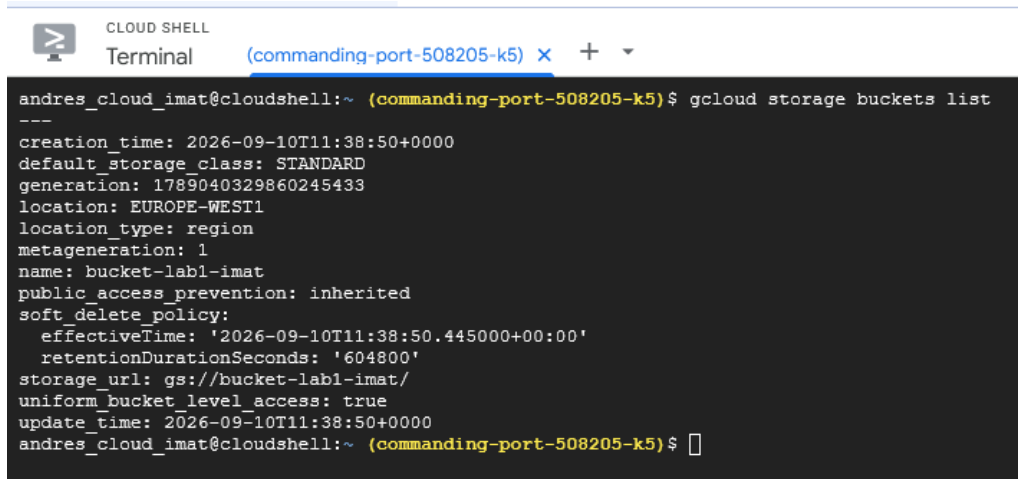
Listing 3.1: Cloud Shell configuration

```
gcloud config list
gcloud config set project commanding-port-508205-k5
```

3.2 Bucket creation

Listing 3.2: Bucket creation and listing

```
gcloud storage buckets create gs://bucket-lab1-imat --location europe-
↪ west1
gcloud storage buckets list
```



```

andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud storage buckets list
---
creation_time: 2026-09-10T11:38:50+0000
default_storage_class: STANDARD
generation: 1789040329860245433
location: EUROPE-WEST1
location_type: region
metageneration: 1
name: bucket-lab1-imat
public_access_prevention: inherited
soft_delete_policy:
  effectiveTime: '2026-09-10T11:38:50.445000+00:00'
  retentionDurationSeconds: '604800'
storage_url: gs://bucket-lab1-imat/
uniform_bucket_level_access: true
update_time: 2026-09-10T11:38:50+0000
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$

```

Figure 11: Properties of the bucket bucket-lab1-imat

Q: What is the bucket location type? What does it mean?

Location type **region**. The bucket lives in one single Google Cloud region, EUROPE-WEST1, where the data is stored redundantly across several zones.

Q: What is the bucket storage class? What does it mean?

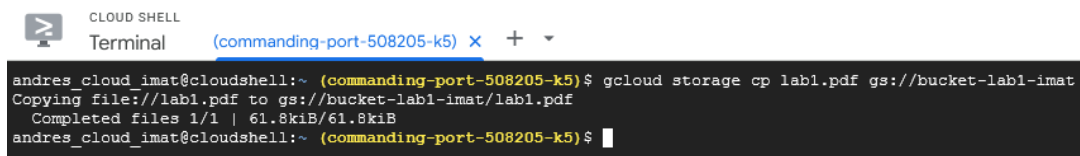
Storage class **STANDARD**. It is meant for frequently accessed data: low latency, high availability and no minimum storage duration, is the most expensive class.

3.3 File upload

The lab statement pdf itself was uploaded to Cloud Shell with the toolbar and then copied to the bucket:

Listing 3.3: Copying the file to the bucket

```
gcloud storage cp lab1.pdf gs://bucket-lab1-imat
```



```

andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud storage cp lab1.pdf gs://bucket-lab1-imat
Copying file:///lab1.pdf to gs://bucket-lab1-imat/lab1.pdf
Completed files 1/1 | 61.8kiB/61.8kiB
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$

```

Figure 12: Copy of lab1.pdf to the bucket

3.4 Browser access and permissions

Opening the object directly from a browser at <https://storage.googleapis.com/bucket-lab1-imat/lab1.pdf> returned **AccessDenied**: the anonymous caller does not have the **storage.objects.get** permission. Uploading a file to a bucket does not make it publicly reachable.

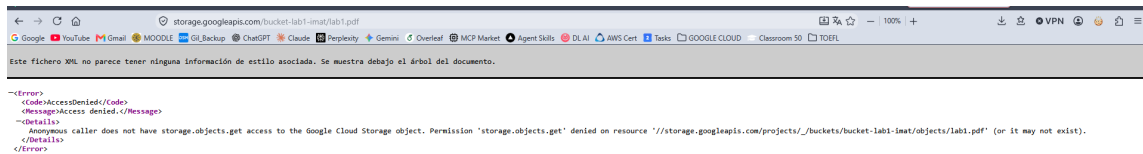


Figure 13: AccessDenied when accessing the object from the browser

In the second project the same test was repeated with the bucket `lab1_bucket_up` and a different file, and the result was the same `AccessDenied` page.

Q: What is needed to access the file?

Authentication and the appropriate IAM permissions, since Cloud Storage objects are not public by default. Alternatively, the object could be made publicly accessible or shared through a signed URL.

4 Task 3 – Custom images

4.1 VM2

Name	vm2
Machine type	e2-medium
Image	Debian 12 (default)
Region / zone	europe-west1 / europe-west1-c
Network tag	deep
Network	ml-network

Table 5: VM2 configuration

4.2 Nginx installation

After connecting with the browser SSH option:

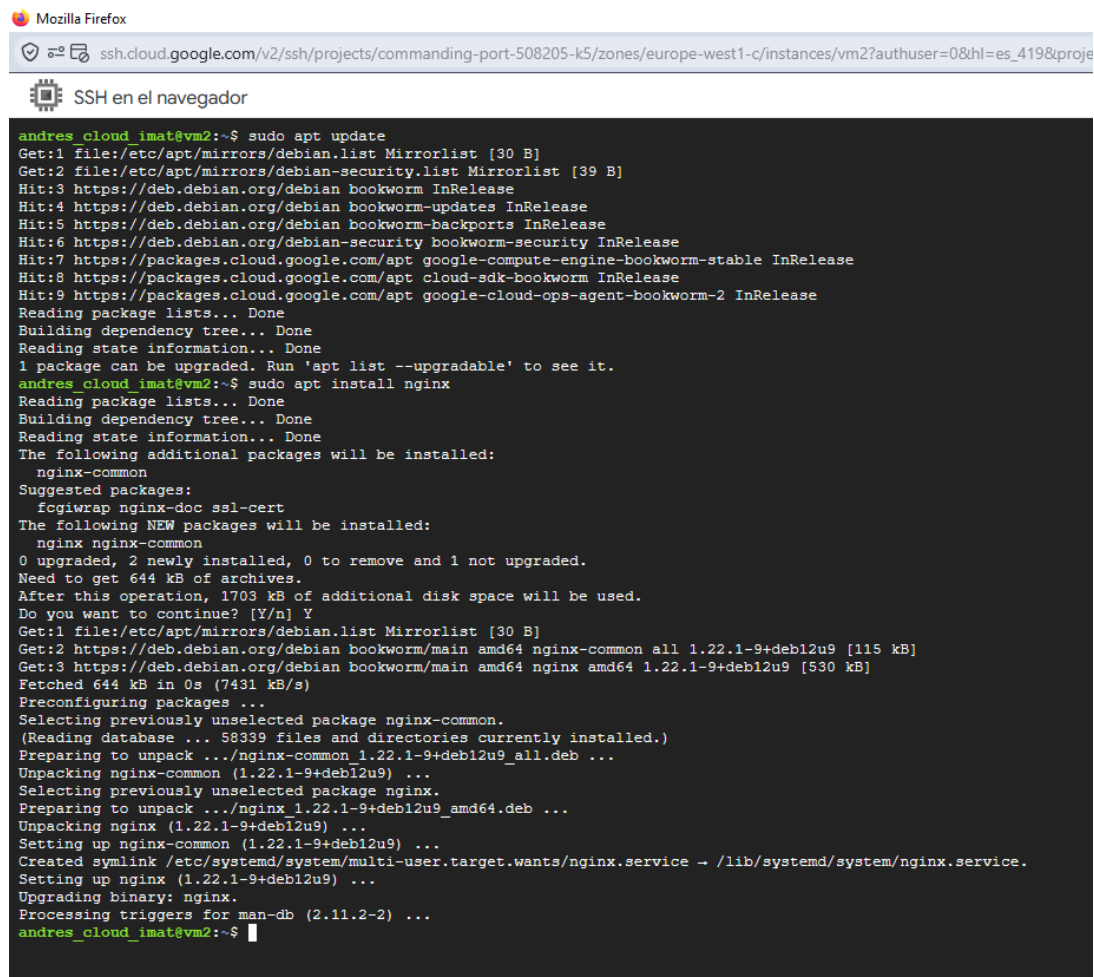
Listing 4.1: Nginx installation on VM2

```

sudo apt update
sudo apt install nginx

```

Nginx 1.22.1 was installed and the service was checked from inside the machine with `curl localhost`, which already returned the default page.



```

Mozila Firefox
ssh.cloud.google.com/v2/ssh/projects/commanding-port-508205-k5/zones/europe-west1-c/instances/vm2?authuser=0&hl=es_419&proj=

SSH en el navegador

andres_cloud_imat@vm2:~$ sudo apt update
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Hit:3 https://deb.debian.org/debian bookworm InRelease
Hit:4 https://deb.debian.org/debian bookworm-updates InRelease
Hit:5 https://deb.debian.org/debian bookworm-backports InRelease
Hit:6 https://deb.debian.org/debian-security bookworm-security InRelease
Hit:7 https://packages.cloud.google.com/apt google-compute-engine-bookworm-stable InRelease
Hit:8 https://packages.cloud.google.com/apt cloud-sdk-bookworm InRelease
Hit:9 https://packages.cloud.google.com/apt google-cloud-ops-agent-bookworm-2 InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
andres_cloud_imat@vm2:~$ sudo apt install nginx
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc ssl-cert
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 1 not upgraded.
Need to get 644 kB of archives.
After this operation, 1703 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 https://deb.debian.org/debian bookworm/main amd64 nginx-common all 1.22.1-9+deb12u9 [115 kB]
Get:3 https://deb.debian.org/debian bookworm/main amd64 nginx amd64 1.22.1-9+deb12u9 [530 kB]
Fetched 644 kB in 0s (7431 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 58339 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.22.1-9+deb12u9_all.deb ...
Unpacking nginx-common (1.22.1-9+deb12u9) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.22.1-9+deb12u9_amd64.deb ...
Unpacking nginx (1.22.1-9+deb12u9) ...
Setting up nginx-common (1.22.1-9+deb12u9) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /lib/systemd/system/nginx.service.
Setting up nginx (1.22.1-9+deb12u9) ...
Upgrading binary: nginx.
Processing triggers for man-db (2.11.2-2) ...
andres_cloud_imat@vm2:~$

```

Figure 14: Nginx installation on VM2

4.3 Browser verification and HTTP firewall rule

The external IP of our VM2 was 35.195.106.18, but `http://35.195.106.18` was not reachable at first because TCP 80 was not allowed by any firewall rule in `ml-network`, so the following rule was created from Cloud Shell:

Listing 4.2: Firewall rule for HTTP traffic

```

gcloud compute firewall-rules create ml-network-allow-http \
  --network=ml-network \
  --direction=INGRESS \
  --action=ALLOW \
  --rules=tcp:80 \
  --source-ranges=0.0.0.0/0

```

The rule allows ingress HTTP traffic towards the instances of the network with the target configuration used. Opening TCP 80 to 0.0.0.0/0 is acceptable here because the point of the exercise is precisely to publish a web server.

This way, we are now able to access the VM2 from our browser and verify the installation of Nginx:

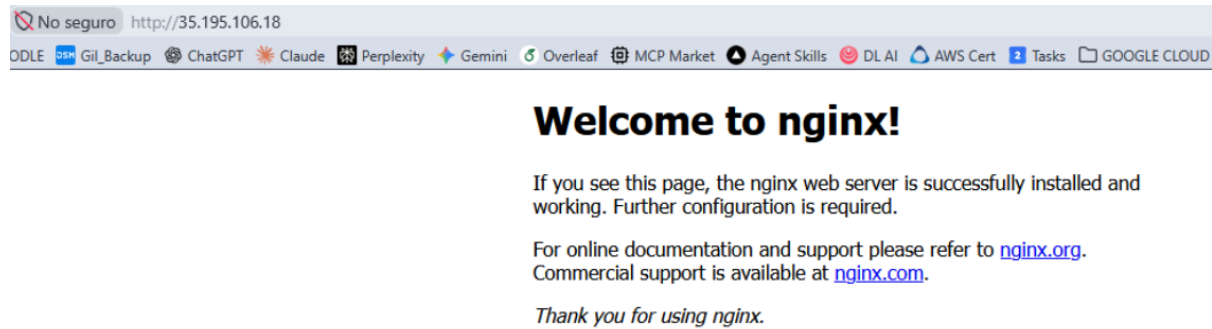


Figure 15: Accessing VM2 from browser

4.4 Custom image

VM2 was stopped, since a running disk cannot be used as image source, and an image of its boot disk was then created with the name **nginx** (945.56 MB).

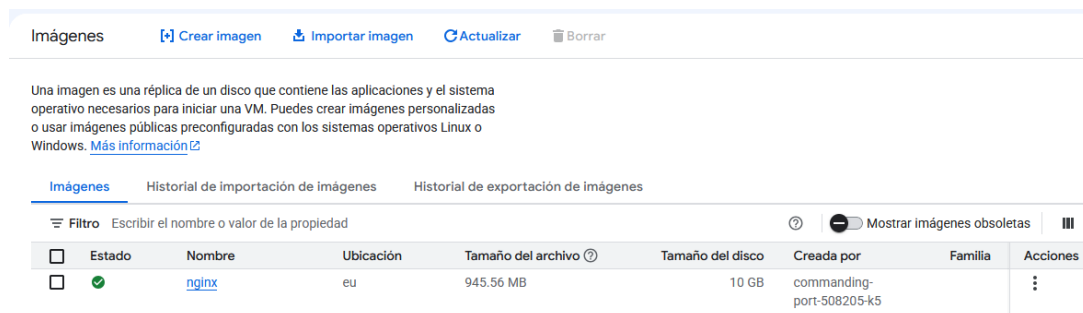
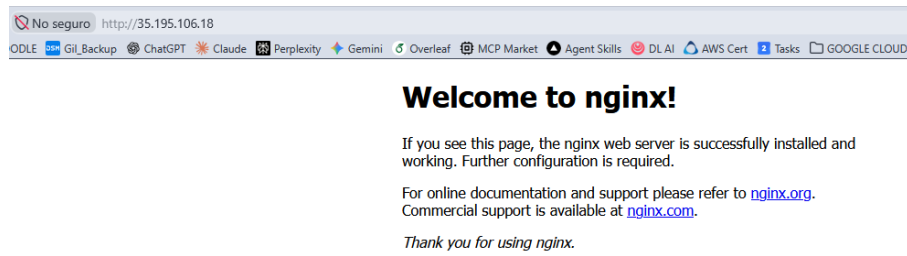


Figure 16: Custom image **nginx** created from the boot disk of VM2

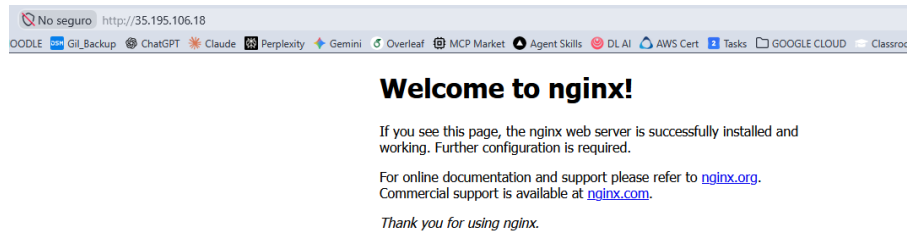
4.5 VM3

VM2 and its boot disk were then deleted, and a new instance **vm3** (same instance config) was created from the custom image **nginx** and attached to **ml-network**.

The goal was to check that Nginx was already installed in the new machine without doing anything else, and it was: the page answered without touching the machine in either project. VM3 got the external IP **35.195.106.18**, the same one previously used by VM2, and pointing the browser at it showed the Nginx page again. No new firewall rule was needed, because **ml-network-allow-http** applies to all the instances of the network.



(a) From VM2, after creating the HTTP rule



(b) From VM3, created from the custom image

Figure 17: Default Nginx page served over HTTP

Filtro Filtrar según nombre de instancia, proyecto o subred				
Nombre ↑	Subred	Rangos de IP internas	Rangos de IP externas	Etiquetas
vm1	ml-subnetwork	10.0.0.2	34.38.162.28	deep
vm3	ml-subnetwork	10.0.0.4	35.195.106.18	deep

Figure 18: Instances affected by `ml-network-allow-http` and their external IPs

4.6 Cleanup

All Compute Engine instances, disks and the custom image were deleted at the end of this task. The network `ml-network` was kept, since it is reused in the following lab practices.

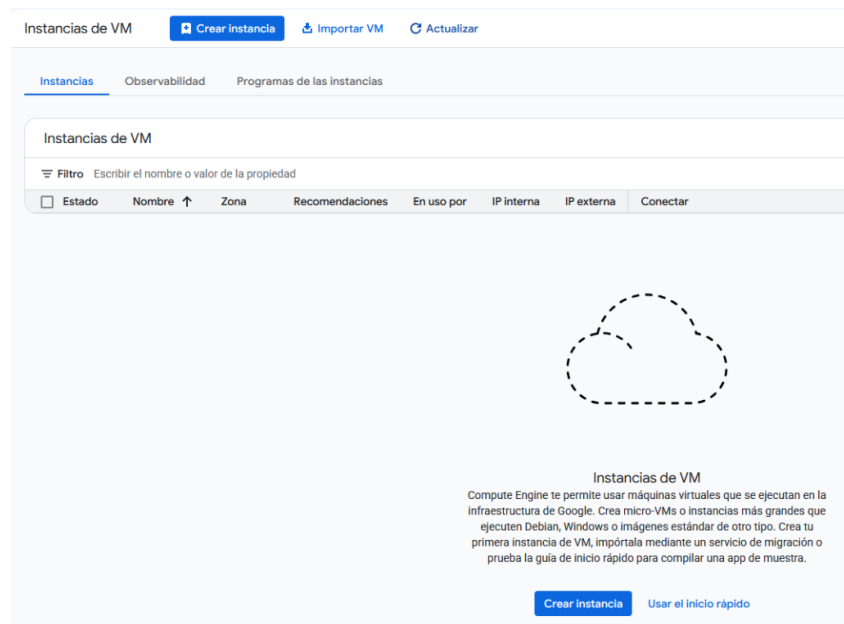


Figure 19: All Compute Engine instances are deleted

5 Task 4 – Google Cloud Marketplace

AI and machine learning solutions found in the Marketplace:

- NVIDIA AI Enterprise
- AI Agents using CrewAI Studio & Jupyter with GPU support
- Red Hat Enterprise Linux AI
- Le Chat Enterprise, an assistant that connects tools and data and allows building agents
- Talk to Your Data, a deployed AI service to query your own data

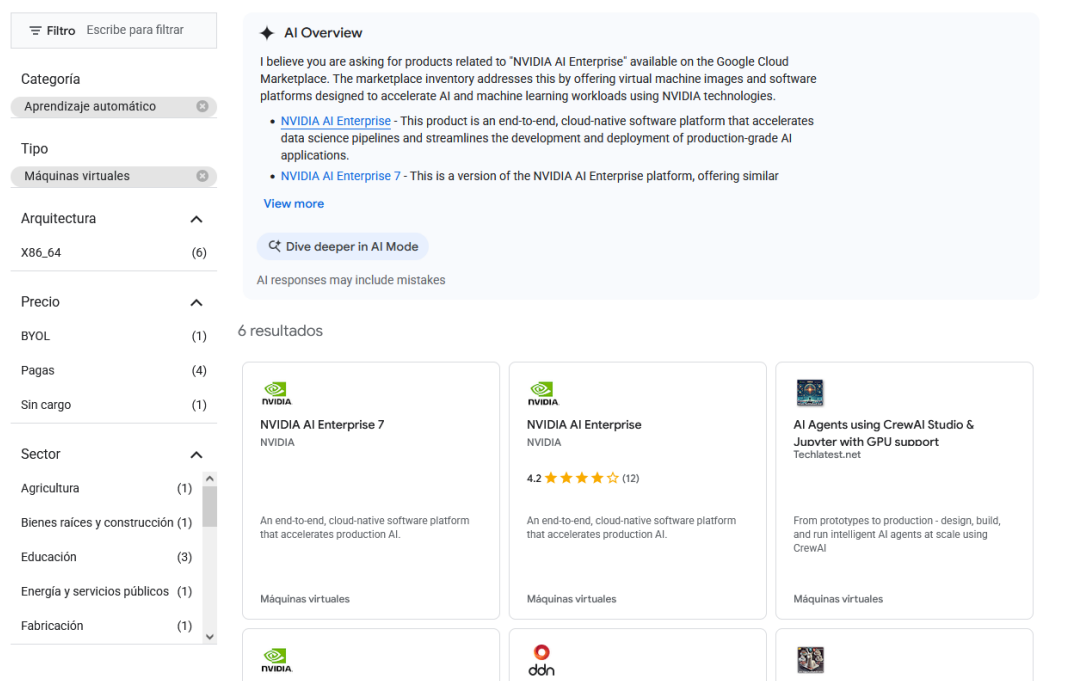


Figure 20: AI and machine learning solutions in the Marketplace

Q: Which are the advantages of using the Marketplace solutions?

They are preconfigured and technically validated, so they can be deployed quickly into an existing Google Cloud environment with little manual configuration of complex AI/ML stacks. They also simplify procurement and licensing, often integrating billing with Google Cloud, and they give access to solutions from established third-party vendors running on Google Cloud infrastructure with its scalability. Some of them offer models that are already deployed and can be queried directly, which saves both infrastructure and training costs.

6 Task 5 – Google Cloud CLI

Task 3 was repeated from Cloud Shell using only `gcloud` commands instead of the console. The commands used are listed below in the order they were executed.

6.1 Environment variables

Listing 6.1: Environment variables

```
PROJECT_ID=$(gcloud config get-value project)

ZONE=europe-west1-c
REGION=europe-west1
NETWORK=ml-network
```

```
SUBNET=$(gcloud compute networks subnets list \
  --network="$NETWORK" \
  --regions="$REGION" \
  --format="value(name)" | head -n1)
```

6.2 Create VM2

Listing 6.2: Creating VM2 from the CLI

```
gcloud compute instances create vm2 \
  --zone="$ZONE" \
  --machine-type=e2-medium \
  --image-family=debian-12 \
  --image-project=debian-cloud \
  --network="$NETWORK" \
  --subnet="$SUBNET" \
  --tags=deep
```

The screenshot shows the Google Cloud Platform interface. At the top, there's a search bar and a 'lab1-a' tab. Below that, the 'CLOUD SHELL' section is active, showing a terminal window. The terminal output shows the command to create VM2 and its details: NAME: vm2, ZONE: europe-west1-c, MACHINE_TYPE: e2-medium, PREEMPTIBLE: false, INTERNAL_IP: 10.0.0.5, EXTERNAL_IP: 34.38.162.28, STATUS: RUNNING.

```

Google Cloud lab1-a
Buscar (/) recursos, documentos, productos y m...

CLOUD SHELL
Terminal (commanding-port-508205-k5) x + v

andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute instances create vm2 \
--zone="$ZONE" \
--machine-type=e2-medium \
--image-family=debian-12 \
--image-project=debian-cloud \
--network="$NETWORK" \
--subnet="$SUBNET" \
--tags=deep
Created [https://www.googleapis.com/compute/v1/projects/commanding-port-508205-k5/zones/europe-west1-c/instances/vm2].
NAME: vm2
ZONE: europe-west1-c
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.0.0.5
EXTERNAL_IP: 34.38.162.28
STATUS: RUNNING
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$
```

Figure 21: Creation of VM2 from the CLI

6.3 Install and verify Nginx

Listing 6.3: Installing and verifying Nginx on VM2

```
gcloud compute ssh vm2 \
  --zone="$ZONE" \
  --tunnel-through-iap \
  --command="sudo apt-get update && sudo apt-get install -y nginx"

gcloud compute ssh vm2 \
  --zone="$ZONE" \
  --tunnel-through-iap \
```

```
--command="systemctl is-active nginx && curl -I http://localhost"
```

The service answered `active` and `curl` returned `HTTP/1.1 200 OK` with `Server: nginx/1.22.1`.

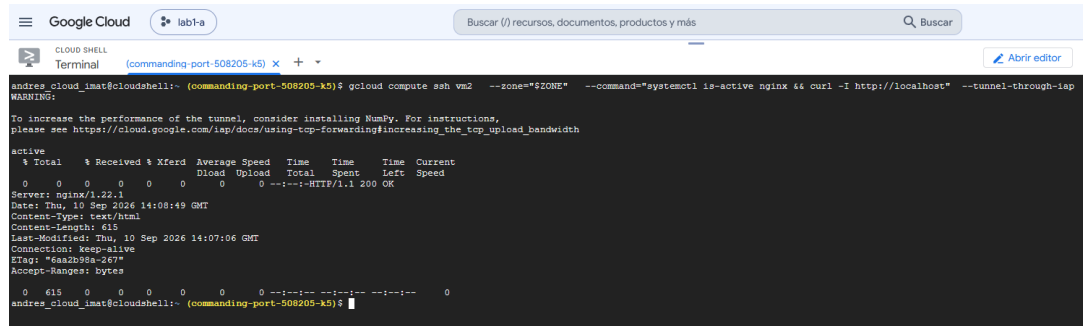


Figure 22: Nginx active on VM2 and answering locally

6.4 Stop VM2 and create the image

Listing 6.4: Stopping VM2 and creating the custom image

```
gcloud compute instances stop vm2 --zone="$ZONE"

gcloud compute images create nginx \
  --source-disk=vm2 \
  --source-disk-zone="$ZONE"
```

```

andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute images create nginx \
--source-disk-vm2 \
--source-disk-zone="$ZONE"
Created [https://www.googleapis.com/compute/v1/projects/commanding-port-508205-k5/global/images/nginx].
NAME: nginx
PROJECT: commanding-port-508205-k5
FAMILY:
DEPRECATED:
STATUS: READY
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$

```

Figure 23: Custom image created from the CLI

6.5 Delete VM2

Listing 6.5: Deleting VM2

```
gcloud compute instances delete vm2 \
  --zone="$ZONE" \
  --delete-disks=all
```

6.6 Create VM3 from the custom image

Listing 6.6: Creating VM3 from the custom image

```
gcloud compute instances create vm3 \
  --zone="$ZONE" \
  --machine-type=e2-medium \
  --image=nginx \
  --image-project="$(gcloud config get-value project)" \
  --network=ml-network \
  --subnet="$SUBNET" \
  --tags=deep
```

```
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute instances create vm3 \
  --zone="$ZONE" \
  --machine-type=e2-medium \
  --image=nginx \
  --image-project="$(gcloud config get-value project)" \
  --network=ml-network \
  --subnet="$SUBNET" \
  --tags=deep
Your active configuration is: [cloudshell-16937]
Created [https://www.googleapis.com/compute/v1/projects/commanding-port-508205-k5/zones/europe-west1-c/instances/vm3].
NAME: vm3
ZONE: europe-west1-c
MACHINE_TYPE: e2-medium
PREEMPTIBLE:
INTERNAL_IP: 10.0.0.6
EXTERNAL_IP: 35.195.106.18
STATUS: RUNNING
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$
```

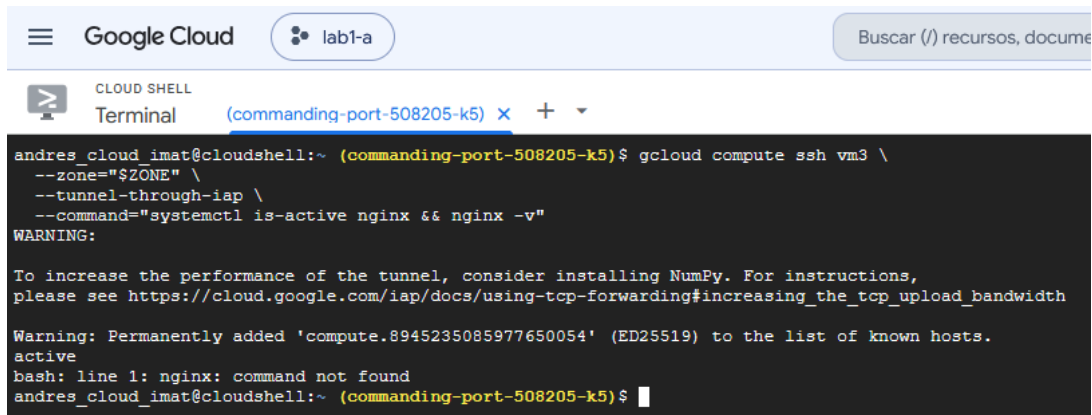
Figure 24: Creation of VM3 from the custom image

6.7 Verify VM3

Listing 6.7: Verifying Nginx on VM3

```
gcloud compute ssh vm3 \
  --zone="$ZONE" \
  --tunnel-through-iap \
  --command="systemctl is-active nginx && nginx -v"
```

The service was reported as `active`, which is what we wanted to check. The second part of the command (for the version) failed with `nginx: command not found`, because the binary is not in the `PATH` of the non-root user; `/usr/sbin/nginx -v` or `sudo` would have been needed. Anyway, we could check what we needed to check.



```

Google Cloud lab1-a
CLOUD SHELL
Terminal (commanding-port-508205-k5) x + v
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute ssh vm3 \
  --zone="$ZONE" \
  --tunnel-through-iap \
  --command="systemctl is-active nginx && nginx -v"
WARNING:
To increase the performance of the tunnel, consider installing NumPy. For instructions,
please see https://cloud.google.com/iap/docs/using-tcp-forwarding#increasing_the_tcp_upload_bandwidth
Warning: Permanently added 'compute.8945235085977650054' (ED25519) to the list of known hosts.
active
bash: line 1: nginx: command not found
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$

```

Figure 25: Nginx already active on VM3

The external IP needed to open the page from the browser was obtained with:

Listing 6.8: External IP of VM3

```

gcloud compute instances describe vm3 \
  --zone="$ZONE" \
  --format="get(networkInterfaces[0].accessConfigs[0].natIP)"

```

6.8 Cleanup

Listing 6.9: Cleanup and verification

```

gcloud compute instances delete vm3 \
  --zone="$ZONE" \
  --delete-disks=all

gcloud compute images delete nginx

gcloud compute instances list
gcloud compute disks list
gcloud compute images list --no-standard-images

```

The three listings returned no items, confirming that everything had been removed.

```

andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute instances list
Listed 0 items.
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute disks list
Listed 0 items.
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$ gcloud compute images list --no-standard-images
Listed 0 items.
andres_cloud_imat@cloudshell:~ (commanding-port-508205-k5)$

```

Figure 26: Resources removed at the end of Task 5

7 Task 6 – Google Cloud API

The source code implementing Task 3 through the Google Cloud API and official SDK is included in the attached deliverables submitted with this report.

8 Conclusions

The practice covered the basic workflow of Google Cloud: creating a custom VPC with its own subnet, restricting SSH access with a firewall rule based on IAP, launching Compute Engine instances (including a Deep Learning VM running JupyterLab through an SSH tunnel), storing a file in a Cloud Storage bucket and checking that objects are not public by default, and building a custom image to create a new machine with the software already installed. The same set of tasks was then reproduced from the Google Cloud CLI, and will also be implemented with the Google Cloud API.